Design and Implementation of a Role-Based Airline Management System using Python and SQLite

Project Report

Department of Mathematical Engineering
Yildiz Technical University/Graduate School of Science And Engineering
Emre Yılmaz, İlayda Ceren Yılmaz, Nilüfer Satenay Kanmaz, Yakup Aktaş, Yusuf Yıldız
Team 4

April 2026

***Abstract*—This report details the development of a role-based Airline Management System (AMS) designed to streamline flight operations and reservation processes. Built using Python, SQLite, and Tkinter, the system implements a robust architecture that distinguishes between passenger and administrative workflows. The application features automatic database initialization, integrity enforcement through SQL triggers, and a comprehensive data model. The results demonstrate an efficient, lightweight desktop solution for managing complex airline data relationships and transactional integrity.**

***Index Terms*—Airline Management, Python, SQLite, Tkinter, Database Design, Role-Based Access Control (RBAC).**

I. Introductİon

The rapid growth of the aviation industry necessitates reliable software solutions to manage flight schedules, passenger reservations, and administrative oversight. The primary objective of this project is to develop a demo application that demonstrates realistic role separation in an airline environment. The system focuses on two distinct user paths: the Passenger flow, centered on flight discovery and booking, and the Admin flow, focused on operational management and request processing.

II. Technologİes Used

The system architecture was selected based on the requirements for a lightweight, dependency-free, and rapid-development environment:

- **Python 3:** Served as the core application logic provider, handling the bridge between the GUI and the database.
- **SQLite:** Chosen for its serverless, file-based nature, allowing for easy deployment and zero-configuration setup.
- **Tkinter:** Utilized for the Graphical User Interface (GUI) to provide a native desktop experience without external library overhead.

III. SQL Features Employed

The following SQL features are employed throughout the system:

- **CREATE TABLE with Constraints:** PRIMARY KEY, FOREIGN KEY, UNIQUE, and NOT NULL constraints enforce schema integrity across all entities.
- **Views:** v_flight_summary and v_request_overview pre-join complex relationships to simplify application queries.
- **Triggers:** BEFORE INSERT triggers abort duplicate seat assignments and overbooking attempts at the database layer.
- **JOIN queries:** INNER JOIN and LEFT JOIN operations link users, flights, and reservation data for search and admin review screens.
- **Aggregate Functions:** COUNT() and GROUP BY measure seat occupancy per flight.
- **Subqueries:** Nested SELECT statements cross-reference capacity against current bookings dynamically.
- **DML Operations:** INSERT, UPDATE, and DELETE statements manage the full lifecycle of flights and reservation requests.

IV. Database Archİtecture and Inİtİalİzatİon

A. Automatic Setup

A key feature of the system is the automatic initialization of the database. Upon the first execution, the application checks for the presence of airline.db. If missing, it executes setup.sql to generate the schema and load seed data.

B. Schema Structure

The relational schema consists of several core entities:

- **Users:** Stores credentials and role assignments (Passenger/Admin).
- **Flights:** Contains schedules, origins, destinations, and capacities.
- **Reservation_Requests:** Managed transactional data between users and flights.

C. Views and Triggers

To enhance performance and enforce business logic, the system utilizes:

- **v_flight_summary:** A view providing aggregated flight data for easy search.
- **v_request_overview:** A consolidated view for administrative review.
- **Triggers:** Automated rules to prevent duplicate seat assignments and overbooking beyond flight capacity.

D. Sample Queries

Seats Remaining per Flight:

```
SELECT flight_id, flight_no, origin, destination, flight_date, flight_time
FROM flights
WHERE destination = 'Istanbul' AND flight_date >= DATE('now')
ORDER BY flight_date, flight_time;
```

Passenger Reservation History:

```
SELECT r.request_id, f.flight_no, f.origin, f.destination, f.flight_date, r.seat_no, r.status, r.created_at
FROM reservation_requests r, flights f
WHERE r.flight_id = f.flight_id AND r.passenger_id = :user_id
ORDER BY r.created_at DESC;
```

V. Roles and Workflows

A. Passenger Workflow

Passengers interact with a user-centric interface designed for efficiency:

- **Flight Search:** Filtering and listing available flights based on destination and time.
- **Booking:** Creating reservation requests including specific seat numbers and personal notes.
- **Tracking:** Monitoring the status (Pending/Approved/Rejected) of their requests.

B. Administrative Workflow

The Admin module provides high-level control over the system state:

- **Flight Management:** Full CRUD (Create, Read, Update, Delete) operations on flight schedules.
- **Request Processing:** Reviewing passenger requests with the ability to approve or reject based on operational criteria.
- **System Maintenance:** Managing demo users and auditing system logs.

VI. ER Model Analysis

The Entity-Relationship (ER) model follows a highly normalized structure. It includes auxiliary entities such as Countries, Airports, Routes, Airfare, and Airplane_Type. This granular approach ensures data consistency and allows the system to scale with more complex attributes like multi-leg routes or tiered pricing models. The relationship between Transactions and Auth_Accounts ensures a clear audit trail for all financial and administrative actions.

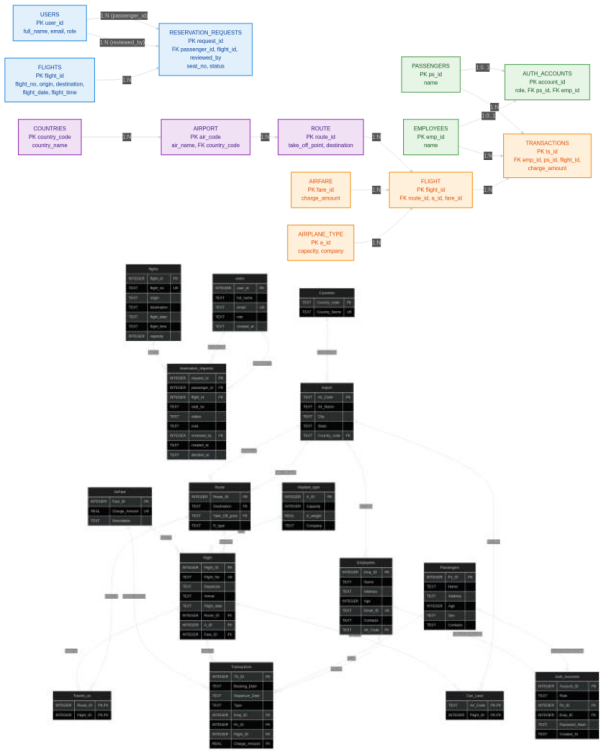

Figure 1. Entity-Relationship Diagram (ERD) of the airline reservation and flight management system.

VI. Conclusion

The Airline Management System project successfully demonstrates the implementation of a role-based desktop application. By leveraging SQLite's trigger system and Python's logic handling, the application maintains high data integrity while providing a responsive user experience. Future iterations could include integration with external APIs for real-time weather tracking and more advanced seat mapping visualizations.